

# Web Platform VPN Capabilities & Browser Extensions - Comprehensive Research

## Web Platform VPN Capabilities & Browser Extensions - Comprehensive Research

**Revision: 1 Last modified:** 2026-07-04T14:00:00Z

**Editorial note (added during the 2026-07-04 MVP2 gap-analysis/hardening pass):** raw research brief preserved as historical input, not a living spec. The final web specification (Proxy Mode / Native Bridge / P2P Mode, Enterprise Hardening detail for extension-store review, enterprise policy push, and auto-update) lives in ../MVP2\_WEB\_CLIENT.md and ../MVP2\_ARCHITECTURE.md §2.2.5 — where this brief's recommendations differ, those documents are authoritative.

### Executive Summary

Web browsers present significant limitations for true VPN functionality due to sandbox constraints: no raw socket access, no TUN/TAP interface creation, and no ability to intercept system-wide traffic. However, browser extensions and modern web APIs can provide partial VPN-like functionality through proxy APIs, WebRTC data channels, WebSocket tunneling, and WebTransport. This research evaluates all web-based approaches for VPN functionality, providing a capability matrix and strategic recommendation for Helix VPN.

**Key Finding:** A pure web-based VPN is impossible for full-device protection. The most viable web strategy is a **hybrid approach**: a native companion app handling the actual VPN tunnel (WireGuard/OpenVPN) with a browser extension acting as the control plane (UI, configuration, split-tunneling rules), connected via Native Messaging.

---

## 1. Browser Extension VPN APIs

### 1.1 Chrome Extension Proxy API (`chrome.proxy`)

The `chrome.proxy` API allows extensions to manage Chrome's proxy settings <sup>1</sup>. It relies on the `ChromeSetting` prototype for getting and setting proxy configuration. However, it has critical limitations:

"Use the `chrome.proxy` API to manage Chrome's proxy settings. This API relies on the `ChromeSetting` prototype of the type API for getting and setting the proxy settings." <sup>2</sup>

**MV3 Impact on Proxy Extensions:** Manifest V3 introduced major breaking changes:

"Chrome's Manifest V3 brings about two key changes: the first is that service workers that are active when needed have replaced long-lived background pages and the second is that the 'webRequest' API has been replaced by the 'declarativeNetRequest' API." <sup>3</sup>

"Previously, proxy extensions used `webRequest` to intercept and modify network requests on the fly... However, the new API under Manifest V3 now requires extensions to declare rules upfront for how they plan on handling network requests." <sup>4</sup>

**Proxy Authentication Crisis in MV3:** The `webRequestBlocking` permission removal broke proxy authentication:

"As a proxy extension developer, this is absolutely maddening. We're forced to choose between auth-less open proxies (bad), or baking in a wacky authentication scheme through a side channel (also bad). MV3 drops in 2.5 months, and will leave tens of millions of proxy extension users unable to use products they paid money for." <sup>5</sup>

The Chromium team later addressed this with the `webRequestAuthProvider` permission:

"Our tentative plan (which is not yet finalized) is that the Manifest V3 version of this capability will require extensions to request a new permission scoped to intercepting authentication requests, but will otherwise allow extensions to handle these requests in a similar manner to how they do in Manifest V2." [6](#)

For MV3 proxy authentication, extensions must now include:

```
{  
  "permissions": [  
    "webRequest",  
    "webRequestAuthProvider"  
  ],  
  "host_permissions": ["<all_urls>"]  
}
```

[7](#)

## 1.2 Chrome vpnProvider API (ChromeOS Only)

Chrome provides a `chrome.vpnProvider` API for implementing a true VPN client, but it is **only available on ChromeOS** and requires enterprise policy installation [8](#):

"Use the `chrome.vpnProvider` API to implement a VPN client." ... "Note: This API is only available to extensions installed by enterprise policy in ChromeOS Managed Guest sessions." [9](#)

The API works as follows:

- Extensions call `createConfig()` to create a VPN configuration
- VPN sessions are managed through `onPlatformMessage` events
- IP packets flow through `sendPacket()` and `onPacketReceived`
- A virtual tunnel connects to ChromeOS's network stack [10](#)

This API is **not available on Windows, macOS, or Linux**, making it unsuitable for a cross-platform desktop VPN strategy.

## 1.3 Firefox WebExtension Proxy API

Firefox offers the most capable browser proxy API through `browser.proxy.onRequest`:

"Use the proxy API to proxy web requests. You can use the `proxy.onRequest` event listener to intercept web requests, and return an object that describes whether and how to proxy them." [11](#)

Key advantages over Chrome:

- **Per-request proxy decisions:** The `proxy.onRequest` approach allows code in the background script to implement dynamic proxy policies with full access to WebExtension APIs <sup>12</sup>
- **proxyAuthorizationHeader:** Firefox supports embedding proxy credentials directly in `ProxyInfo`, unlike Chrome's more limited approach <sup>13</sup>
- **MV3 support:** Firefox continues to support `webRequestBlocking` in Manifest V3 and provides `webRequestAuthProvider` for cross-browser compatibility <sup>14</sup>

Firefox example:

```
browser.proxy.onRequest.addListener((requestInfo) => {  
  return {type: "http", host: "proxy.example.com", port: 8080};  
}, {urls: ["<all_urls>"]});
```

## 1.4 Safari Web Extensions

Safari extensions have the most limited VPN-relevant capabilities:

- Safari uses system-wide proxy settings only <sup>15</sup>
  - No equivalent to Chrome's `chrome.proxy` or Firefox's `proxy.onRequest` <sup>16</sup>
  - Safari's WebRTC behavior differs: it filters out host candidates (local IPs) by default but can still expose public IPs via STUN <sup>17</sup>
  - Safari Web Extensions on iOS are further constrained by Apple's platform restrictions
- 

# 2. WebRTC as VPN Transport

## 2.1 Architecture Overview

WebRTC data channels provide a browser-native encrypted transport that could theoretically serve as a VPN tunnel. The protocol stack is:

Application Data

- > SCTP (Stream Control Transmission Protocol)
- > DTLS (Datagram TLS)
- > UDP (User Datagram Protocol)
- > IP

Every WebRTC connection is encrypted via DTLS with certificate fingerprint verification <sup>18</sup>:

"Authentication of the remote party is achieved through DTLS certificate fingerprint verification... This is why secure signaling channel is crucial - if fingerprint and ICE info are delivered accurately, the peer connection is secure." <sup>19</sup>

## 2.2 Snowflake: A Working WebRTC Circumvention System

Snowflake (part of the Tor Project) is the most mature real-world implementation of WebRTC-based traffic tunneling:

"Snowflake uses a stack of nested protocol layers... The point-to-point link between a client and its proxy is a WebRTC data channel. Data channels let two WebRTC peers exchange arbitrary binary messages." <sup>20</sup>

Snowflake's protocol stack <sup>21</sup>:

UDP

- > WebRTC data channel (ephemeral, per proxy)
- > DTLS
- > SCTP
  - > KCP + smux (persistent, per session)
  - > Tor protocol
  - > application streams

Key design insights from Snowflake:

- Uses Turbo Tunnel pattern for session persistence across proxy changes
- WebRTC data channels are ephemeral outer layers; inner layers maintain state
- Proxy acts as a dumb relay copying data between WebRTC and WebSocket

## 2.3 WebRTC VPN Limitations

**Complexity:** WebRTC is extremely complex for simple VPN use cases:

"The full set of protocols needed to implement WebRTC is daunting... WebRTC is an over-engineered Rube Goldberg machine... You really, really don't need all that." <sup>22</sup>

**Not a raw UDP socket:** WebRTC data channels add SCTP and DTLS overhead on top of UDP:

"WebRTC data channels use a stack of protocols... UDP for network transport, DTLS for confidentiality and integrity, and SCTP for delimiting message boundaries." <sup>23</sup>

**IP address exposure (WebRTC leak):** Browser-extension VPNs cannot prevent WebRTC leaks because:

"Browser-extension VPNs only proxy HTTP and HTTPS traffic and do not route UDP at all, so they will never prevent WebRTC leaks regardless of other settings." <sup>24</sup>

**Server infrastructure required:** WebRTC requires STUN/TURN servers for NAT traversal:

- STUN: Helps devices discover public IP (~60-70% of connections) <sup>25</sup>
- TURN: Relay fallback for restrictive networks (adds latency, costs bandwidth) <sup>26</sup>

**Maximum message size:** There are practical limits on data channel message sizes:

"Only unfragmented SCTP packets are handled, so any message large enough to cause an SCTP packet to need fragmentation causes an error... The maximum message length depends on the particular browser you connect with, but in my testing currently it is slightly smaller than 1200 bytes." <sup>27</sup>

## 2.4 Security Considerations

WebRTC provides strong encryption but has fingerprinting risks:

- DTLS 1.3 is now mandatory (Feb 2025), providing Perfect Forward Secrecy <sup>28</sup>
  - mDNS obfuscation hides local IP addresses in ICE candidates <sup>29</sup>
  - Enterprise policies can force relay-only mode <sup>30</sup>
  - TURN servers must be secured against relay abuse <sup>31</sup>
-

## 3. WebSocket Tunneling

### 3.1 WebSocket as VPN Transport

WebSocket (specifically `wss://`) is the most practical browser-based tunneling transport. The `wstunnel` project demonstrates a production-quality implementation:

"Tunnel all your traffic over Websocket or HTTP2 - Bypass firewalls/DPI - Static binary available" [32](#)

**Key capabilities of `wstunnel`** [33](#):

- TCP, UDP, SOCKS5, and HTTP proxy forwarding
- Transparent proxy support (Linux only with `CAP_NET_ADMIN`)
- Reverse tunneling
- WireGuard traffic tunneling
- TLS with SNI override for stealth

### 3.2 Practical Browser Integration

The NetBird browser client demonstrates how WebSocket + WASM can enable VPN-like functionality:

"The WASM client shares the same codebase as native NetBird clients but is adapted for the browser environment. All traffic routes through NetBird relay servers using WebSocket while maintaining end-to-end WireGuard encryption." [34](#)

NetBird's multi-layer encapsulation [35](#):

Application Layer (SSH/RDP)

- > TCP Layer
  - > WireGuard Layer (encrypted)
    - > Relay Protocol
      - > WebSocket Layer
        - > TLS Layer
          - > Standard Network Stack

### 3.3 Performance Benchmarks

WebSocket tunneling adds protocol overhead:

- WebSocket frame overhead: 2-14 bytes per frame
- TLS overhead: ~5-20% bandwidth increase
- Typical latency increase: 1-3 RTTs for WebSocket handshake + TLS

For comparison, wstunnel achieves near-gigabit throughput in Rust [36](#).

"More throughput and less jitter... You can now saturate a gigabit ethernet card with a single connection" [37](#)

### 3.4 Limitations

- **TCP meltdown problem:** Tunneling TCP over TCP (WebSocket over TCP) causes performance degradation under lossy conditions
  - **No UDP in WebSocket:** WebSocket is TCP-only; UDP-like behavior must be simulated on top
  - **HTTP upgrade requirement:** Every connection requires an HTTP upgrade handshake
  - **Buffering by intermediaries:** Reverse proxies/CDNs may buffer the entire request, breaking real-time tunneling [38](#)
- 

## 4. WebTransport API

### 4.1 Overview

WebTransport is a modern browser API built on HTTP/3 over QUIC, providing both reliable streams and unreliable datagrams [39](#).

"It enables reliable transport via streams and unreliable transport via UDP-like datagrams." [40](#)

Key characteristics [41](#) [42](#):

- Built on HTTP/3 over QUIC
- Provides reliable bidirectional streams (like WebSocket but better)
- Provides unreliable datagrams (UDP-like, but encrypted and congestion-controlled)
- Supports 0-RTT connection resumption
- Better network transition handling (WiFi to cellular)
- No head-of-line blocking across streams

### 4.2 Datagrams for VPN-like Use Cases

WebTransport datagrams are the closest browsers get to raw UDP:

```
const transport = new WebTransport("https://vpn.example.com:443");
const writer = transport.datagrams.writable.getWriter();
```



```

const reader = transport.datagrams.readable.getReader();

// Send datagram
writer.write(new Uint8Array([/* IP packet data */]));

// Receive datagram
const {value, done} = await reader.read();

```

43

Important caveat from Google:

"No. WebTransport is not a UDP Socket API. While WebTransport uses HTTP/3, which in turn uses UDP 'under the hood,' WebTransport has requirements on encryption and congestion control, so it is not simply a basic UDP socket API." <sup>44</sup>

### 4.3 VPN Applicability Assessment

| Aspect                | WebTransport                            | Ideal VPN Transport           |
|-----------------------|-----------------------------------------|-------------------------------|
| Unreliable datagrams  | Yes (encrypted + congestion-controlled) | Yes (raw UDP)                 |
| Connection-oriented   | Yes (QUIC connection required)          | No (stateless preferred)      |
| Encryption            | Mandatory (TLS 1.3)                     | Optional (WireGuard/ChaCha20) |
| Raw IP packet support | No                                      | Yes                           |
| Browser support       | Chrome, Firefox (growing)               | N/A                           |

**Verdict:** WebTransport datagrams could serve as the transport for a custom VPN protocol within the browser, but cannot replace raw UDP for standard VPN protocols like WireGuard or OpenVPN.

### 4.4 WebTransport vs WebSocket for VPN

| Feature               | WebSocket | WebTransport           |
|-----------------------|-----------|------------------------|
| Protocol              | TCP       | HTTP/3 (QUIC)          |
| Unreliable delivery   | No        | Yes (datagrams)        |
| Multiplexing          | No        | Yes (multiple streams) |
| 0-RTT                 | No        | Yes                    |
| Head-of-line blocking | Yes       | No (across streams)    |

| Feature          | WebSocket | WebTransport    |
|------------------|-----------|-----------------|
| Server ecosystem | Mature    | Emerging        |
| Browser support  | Universal | Chrome, Firefox |

[45](#) [46](#)

## 5. Secure Web Proxy / HTTPS Proxy

### 5.1 HTTP CONNECT Method

The HTTP CONNECT method creates TCP tunnels through proxies:

"A CONNECT request tells an HTTP proxy to establish a TCP connection to the specified host and port, then relay raw bytes in both directions." [47](#)

The sequence [48](#):

1. Client sends CONNECT example.com:443 HTTP/1.1 to proxy
2. Proxy opens TCP connection to example.com:443
3. Proxy responds with 2xx success
4. Client performs TLS handshake directly with destination
5. All subsequent traffic passes as opaque byte stream

### 5.2 Limitations vs Full VPN

#### What HTTPS proxy CAN do:

- Encrypt browser HTTP/HTTPS traffic
- Route TCP traffic through a remote server
- Authenticate users per-connection

#### What HTTPS proxy CANNOT do:

- Route UDP traffic (unless using SOCKS5 proxy)
- Provide system-wide protection
- Intercept non-browser traffic
- Handle arbitrary IP protocols (ICMP, etc.)
- Provide a kill switch

"The WHATWG Fetch Standard lists CONNECT as a forbidden method, preventing JavaScript from issuing CONNECT requests through browser APIs." [49](#)

This is a critical limitation: JavaScript cannot directly establish CONNECT tunnels. Browser extensions must use the proxy API instead.

## 5.3 HTTP/2 and HTTP/3 Proxy

Extended CONNECT protocol (RFC 8441) enables tunneling over HTTP/2:

"In HTTP/2 and HTTP/3, CONNECT creates a tunnel over a single stream rather than the full TCP connection." [50](#)

This allows multiplexed tunnels but is not widely supported for general proxy use.

---

## 6. PWA VPN Limitations

### 6.1 Service Worker Network Interception

Progressive Web Apps use Service Workers for offline functionality, but their network interception capabilities are limited:

#### What Service Workers CAN intercept:

- HTTP requests made from the PWA's scope (same-origin + registered path)
- fetch events for controlled pages
- Subresource requests (images, scripts, stylesheets)

#### What Service Workers CANNOT do:

- Intercept traffic from other origins (unless CORS allows)
- Intercept traffic from other applications
- Create raw sockets or TUN interfaces
- Intercept WebSocket connections (can only observe, not intercept)
- Modify proxy settings at the OS level
- Handle UDP traffic

### 6.2 PWA as VPN Companion App

A PWA could serve as a **control interface** for a VPN but cannot implement the tunnel itself:

- Display connection status and server selection
- Manage user settings and preferences

- Communicate with a native app via Native Messaging (requires browser extension bridge)
- Use the Badging API for connection status indicators

**Critical limitation:** PWAs cannot register as VPN clients, cannot create persistent tunnels, and cannot intercept system traffic. The PWA model is fundamentally unsuited for VPN functionality beyond UI/control.

---

## 7. WebAssembly for VPN Cryptography

### 7.1 WASM Performance for Crypto

Research shows WebAssembly significantly outperforms JavaScript for cryptographic operations:

"WebAssembly significantly outperformed JavaScript in single-operation tests for both encryption and decryption. While JavaScript's encryption time was approximately fivefold slower than that of WebAssembly, its decryption performance lagged by about 4.5 times." <sup>51</sup>

Specific benchmarks <sup>52</sup>:

- RSA 2048-bit encryption: JS 6.14ms vs WASM 1.23ms (**5x faster**)
- RSA 2048-bit decryption: JS 28.87ms vs WASM 6.59ms (**4.4x faster**)
- Ed25519 batch verification: JS 8ms vs WASM 0.07ms (**~114x faster**)

### 7.2 WebAssembly vs WebCrypto API

The Web Cryptography API (native browser implementation) is faster than WASM for some operations:

"Using the Web Crypto API will take 1.4 seconds on average for a single key pair [RSA 4096]. The same task takes 6.3 seconds on average when using Botan (as WASM)." <sup>53</sup>

Best practice: Use WebCrypto for standard operations (AES-GCM, RSA, ECDH) and WASM only for:

- Custom/WireGuard-specific protocols not supported by WebCrypto
- Post-quantum cryptography algorithms
- Batch operations where algorithmic optimization matters

## 7.3 Real-World WASM VPN: NetBird Browser Client

NetBird demonstrates a production browser VPN client using WASM:

"The Browser Client runs two WASM modules: the NetBird client (which handles SSH, WireGuard tunneling, and networking) and IronRDP (for RDP protocol handling)." [54](#)

Architecture [55](#):

- **Go WASM module:** NetBird peer compiled to WASM using Go's native WASM support
- **WireGuard:** Uses standard wireguard-go
- **Transport:** All traffic routes through NetBird relay servers using WebSocket
- **Rust WASM module:** IronRDP for RDP protocol handling

## 7.4 Tailscale WASM

Tailscale has also been ported to WASM for browser use:

"To make this possible, we ported the following to WebAssembly: the Tailscale client, WireGuard, a complete userspace network stack (from gVisor), and an SSH client." [56](#)

"By slightly modifying their browser ssh client (tsconnect), and implementing a custom Tun device, we successfully communicated with a Tailscale network by just sending/receiving IP packets on a JavaScript MessageChannel!" [57](#)

Challenge: The compiled WASM module is 16MB, which is significant for web delivery [58](#).

---

# 8. Shadowsocks Web / Browser Implementation

## 8.1 Outline Client Architecture

Outline (developed by Jigsaw/Google) is the most prominent Shadowsocks-based VPN:

"Outline clients use the popular Shadowsocks protocol, and lean on the Cordova and Electron frameworks to support Windows, Android/ChromeOS, Linux, iOS and macOS." [59](#)

Key architecture details <sup>60</sup>:

- UI implemented in Polymer 2.0 (web technologies)
- Platform support via Cordova (mobile) and Electron (desktop)
- Additional native components for tunneling
- Shared web app code in `src/www` directory
- Browser platform development uses fake servers for testing

## 8.2 Shadowsocks in Browser Context

Shadowsocks is a SOCKS5 proxy protocol, not a true VPN:

"Outline is not a true VPN. Rather it uses an open-source SOCKS5 proxy called Shadowsocks which is designed to protect your Internet traffic." <sup>61</sup>

The Outline client applications use OS VPN APIs to route all traffic through the Shadowsocks proxy, creating the appearance of a VPN. In a pure browser context:

- Shadowsocks cannot be implemented without a native host for the actual tunnel
- Browser extensions can configure the browser to use a Shadowsocks proxy (via `chrome.proxy`)
- The encryption/decryption must happen in a native component or WASM

## 8.3 Web-based Shadowsocks Limitations

- Requires a native SOCKS5 listener (browsers cannot create raw UDP/TCP listeners)
  - The AEAD cipher implementations could run in WASM but still need a network endpoint
  - Shadowsocks' active probing resistance requires low-level packet timing that WASM cannot provide
- 

# 9. Major VPN Browser Extension Architectures

## 9.1 VPN Extension vs App: Fundamental Distinction

Most "VPN browser extensions" are actually **proxies**, not VPNs:

"Many 'VPN' extensions are essentially proxies, indeed: they cannot affect anything except the traffic of the browser itself (through `chrome.proxy` and `chrome.webRequest` API)." <sup>62</sup>

Two exceptions exist <sup>63</sup>:

1. ChromeOS: `chrome.vpnProvider` API for true VPN
2. Native Messaging: Extension communicates with a native app that manages the actual VPN

## 9.2 Windscribe Architecture

Windscribe provides a clear example of the distinction:

"The desktop app provides a full VPN tunnel for all your computer's traffic while the extension is meant to serve as a lightweight but secure browser proxy and offer additional privacy features that can't be implemented via a VPN application." <sup>64</sup>

Recommended dual setup <sup>65</sup>:

- Desktop app: VPN tunnel for all traffic + firewall
- Browser extension: Ad/tracker blocking, cookie management, user-agent spoofing
- Combined: Extension + App = Double Hop mode

## 9.3 NordVPN / ExpressVPN / ProtonVPN Extensions

Major VPN providers typically offer:

- **Browser extension:** HTTP/HTTPS proxy using `chrome.proxy` API, with optional WebRTC leak blocking
- **Desktop app:** System-level WireGuard/OpenVPN tunnel
- **Integration:** Extension can detect and communicate with the desktop app (sometimes via Native Messaging)

## 9.4 Browser Extension Security Concerns

"Extensions can see your browser traffic. They must be trusted. A malicious extension could log URLs, cookies, or form data." <sup>66</sup>

Common failure modes <sup>67</sup>:

- Silent disconnection without notice
- WebRTC IP leaks (extensions only proxy HTTP/HTTPS)
- DNS leaks (may route DNS through ISP)

- No kill switch for browser-level VPNs
- 

## 10. Native Messaging for Browser Extensions

### 10.1 Architecture Overview

Native Messaging allows browser extensions to communicate with native applications:

"Native messaging is a Web-to-App communication mechanism supported in all modern browsers (Firefox, Chrome, Edge) to exchange UTF8-encoded JSON messages between a browser extension and a native host application."  
[68](#)

Communication flow [69](#):

1. Extension calls `chrome.runtime.connectNative()` or `chrome.runtime.sendNativeMessage()`
2. Browser spawns the native host process
3. Messages exchanged via stdin/stdout (UTF-8 JSON with 32-bit length prefix)
4. Process terminates when port disconnects

### 10.2 Security Model

"Because Native Messaging is only working in the bounds of the machine the security risk is minimal when the host was carefully implemented. An attacker has to be already on the machine to be able to abuse it." [70](#)

Security characteristics:

- Native host runs outside browser sandbox with full OS permissions
- Extension must declare `nativeMessaging` permission
- Host manifest specifies `allowed_origins` (which extensions can connect)
- No network exposure - local only
- Max message size: 1 MB (host), 4 GB (extension) [71](#)

### 10.3 Cross-Platform Host Manifest

**Chrome/Edge** (registry key on Windows) [72](#):



```
{
  "name": "com.helix.vpn.host",
  "description": "Helix VPN Native Host",
  "path": "C:\\\\Program Files\\\\Helix VPN\\\\helix-host.exe",
  "type": "stdio",
  "allowed_origins": ["chrome-extension://EXTENSION_ID/"]
}
```

**Firefox** (file-based manifest) <sup>73</sup>:

```
{
  "name": "com.helix.vpn.host",
  "description": "Helix VPN Native Host",
  "path": "/usr/bin/helix-host",
  "type": "stdio",
  "allowed_extensions": ["helix@example.org"]
}
```

## 10.4 VPN-Specific Use Cases

Native Messaging enables:

- **Extension controls native VPN:** Browser UI for connecting/disconnecting, server selection
  - **Status reporting:** Native app reports tunnel state, IP address, data usage
  - **Split tunneling rules:** Extension sends per-site rules to native app
  - **Credential management:** Secure storage of VPN credentials in native keychain
  - **Privilege escalation:** Native app handles TUN interface creation, firewall rules
- 

# 11. Split Tunneling in Browsers

## 11.1 PAC Scripts (Proxy Auto-Config)

Proxy Auto-Configuration files enable per-site proxy routing:

```
function FindProxyForURL(url, host) {
  // Direct connection for local/intranet sites
  if (isPlainHostName(host) ||
      shExpMatch(host, "*.local") ||
      isInNet(host, "10.0.0.0", "255.0.0.0") ||
      isInNet(host, "192.168.0.0", "255.255.0.0")) {
    return "DIRECT";
  }
  // Specific domains through proxy
}
```

```

    if (shExpMatch(host, "*.streaming-service.com") ||
        dnsDomainIs(host, "geo-blocked.site")) {
        return "PROXY proxy.vpn.example.com:8080";
    }
    // Default: direct connection
    return "DIRECT";
}

```

## 11.2 Per-Site Proxy Configuration

**Firefox** (most capable) <sup>74</sup>:

```

browser.proxy.onRequest.addListener((requestInfo) => {
    if (requestInfo.url.includes("internal.company.com")) {
        return {type: "direct"};
    }
    return {
        type: "http",
        host: "vpn-proxy.example.com",
        port: 8080,
        proxyAuthorizationHeader: "Basic " + btoa("user:pass")
    };
}, {urls: ["<all_urls>"]});

```

**Chrome** (more limited):

- chrome.proxy settings apply globally to all browser traffic
- PAC script is the only mechanism for per-site routing
- Cannot dynamically change proxy per-request in MV3 without reloading PAC script

## 11.3 Enterprise Deployment

PAC files can be deployed at the OS level [<sup>325</sup>]:

- Windows: Group Policy, Settings app
- macOS: Network preferences
- Linux: Environment variables, NetworkManager
- ChromeOS: Device policy

Cloudflare Gateway example [<sup>325</sup>]:

"Chromium-based browsers (Google Chrome, Microsoft Edge, Brave) and Safari use the operating system proxy settings. Firefox uses its own proxy settings by default and must be configured separately."

---

# 12. Web UI for Native VPN (Tauri vs Electron)

## 12.1 Tauri: Rust + Native WebView

Tauri uses the OS native webview and Rust backend:

"Instead of bundling Chromium, it uses the operating system's native webview (WebKit on macOS, WebView2 on Windows, WebKitGTK on Linux). Instead of a Node.js backend, it uses Rust." <sup>75</sup>

Key metrics <sup>76</sup>:

- Binary size: ~5-20 MB (vs 100-200 MB for Electron)
- Memory: 30-60 MB at idle (vs 150-300 MB for Electron)
- Native performance for backend operations
- No JavaScript garbage collection pauses

## 12.2 Electron: Node.js + Chromium

Electron bundles a complete Chromium browser and Node.js runtime:

"Electron's main process runs as a NodeJS process. This architecture necessitates shipping a Node.js runtime with your app." <sup>77</sup>

Trade-offs <sup>78</sup>:

- Larger bundle size (~100-200 MB)
- Higher memory consumption
- More mature ecosystem
- JavaScript backend (easier for web developers)
- Automatic cross-platform consistency (bundles same Chromium everywhere)

## 12.3 Comparison for VPN Applications

| Aspect           | Tauri                 | Electron           |
|------------------|-----------------------|--------------------|
| Bundle size      | ~5-20 MB              | ~100-200 MB        |
| Memory (idle)    | 30-60 MB              | 150-300 MB         |
| Backend language | Rust                  | JavaScript/Node.js |
| VPN integration  | Native Rust libraries | Node.js bindings   |
| Auto-updater     | Built-in              | Built-in           |
| Code signing     | Supported             | Supported          |

| Aspect          | Tauri           | Electron         |
|-----------------|-----------------|------------------|
| WebRTC handling | Native (WebKit) | Bundled Chromium |

### 12.4 Recommendation

For a VPN application, **Tauri is strongly preferred:**

- Rust backend can directly integrate with WireGuard implementations (wireguard-rs, boringtun)
- Smaller bundle size improves user acquisition
- Lower memory usage means less interference with VPN performance
- Native system tray, notifications, and firewall integration

## 13. Capability Matrix: Web Platform x VPN Feature

| VPN Feature                      | Browser Extension        | WebRTC                      | WebSocket             | WebTransport        | PWA                |
|----------------------------------|--------------------------|-----------------------------|-----------------------|---------------------|--------------------|
| Intercept browser HTTP/HTTPS     | Possible (proxy API)     | Not applicable              | Not applicable        | Not applicable      | Limited (SW scope) |
| Intercept browser UDP            | Impossible               | Impossible (DTLS/SCTP only) | Impossible (TCP only) | Limited (datagrams) | Impossible         |
| System-wide traffic interception | Impossible               | Impossible                  | Impossible            | Impossible          | Impossible         |
| Create TUN interface             | Impossible               | Impossible                  | Impossible            | Impossible          | Impossible         |
| Encrypt traffic end-to-end       | Limited (TLS to proxy)   | Possible (DTLS)             | Possible (WSS)        | Possible (QUIC+TLS) | Same as browser    |
| Kill switch                      | Impossible               | Impossible                  | Impossible            | Impossible          | Impossible         |
| DNS leak protection              | Limited (extension only) | Limited                     | Limited               | Limited             | Impossible         |
| WebRTC leak protection           | Possible (block WebRTC)  | N/A                         | N/A                   | N/A                 | Impossible         |

| VPN Feature                       | Browser Extension       | WebRTC     | WebSocket                | WebTransport             | PWA        |
|-----------------------------------|-------------------------|------------|--------------------------|--------------------------|------------|
| <b>Split tunneling (per-site)</b> | Possible (PAC/Firefox)  | Impossible | Impossible               | Impossible               | Impossible |
| <b>SOCKS5 proxy support</b>       | Limited (browser proxy) | Impossible | Impossible               | Impossible               | Impossible |
| <b>Custom VPN protocol</b>        | Impossible              | Impossible | Limited (over WebSocket) | Limited (over datagrams) | Impossible |
| <b>No additional install</b>      | Yes                     | Yes        | Yes                      | Yes                      | Yes        |

### Legend:

- **Possible:** Fully achievable
- **Limited:** Achievable with significant constraints
- **Impossible:** Blocked by browser security model

## 14. Web-based VPN Approaches: Detailed Assessment

### 14.1 Pure Browser Extension (Proxy-Only)

**How it works:** Uses `chrome.proxy` or `browser.proxy.onRequest` to route browser HTTP/HTTPS traffic through a remote proxy server.

#### Pros:

- No installation required beyond extension
- Cross-platform (any OS with browser support)
- Quick connect/disconnect
- Works in restricted environments (schools, workplaces)

#### Cons:

- Only browser traffic protected
- Cannot route UDP traffic (no true VPN tunnel)
- WebRTC leaks real IP address <sup>79</sup>
- DNS queries may leak to ISP
- No kill switch
- Chrome MV3 limitations for proxy auth

**Verdict:** Suitable for geo-unblocking only, NOT for privacy/security.

## 14.2 WebRTC-based VPN

**How it works:** Establishes WebRTC data channel to a server, encapsulates traffic through DTLS/SCTP.

**Pros:**

- Native browser encryption (DTLS)
- NAT traversal built-in (ICE/STUN/TURN)
- Used successfully by Snowflake for censorship circumvention
- UDP-like transport available

**Cons:**

- Extremely complex protocol stack
- Requires STUN/TURN infrastructure
- Not a true VPN (no TUN interface, no system routing)
- Message size limitations (~1200 bytes unfragmented) <sup>80</sup>
- Server-side requires WebRTC implementation
- Fingerprinting concerns

**Verdict:** Over-engineered for VPN use case. Only viable for specific circumvention scenarios like Snowflake.

## 14.3 WebSocket + WASM VPN

**How it works:** Compile VPN client (WireGuard) to WASM, transport encrypted packets over WebSocket connection to a relay server.

**Pros:**

- Real end-to-end encryption (WireGuard in WASM)
- Works in any modern browser
- No browser extension required
- Can reuse existing VPN codebase (NetBird, Tailscale demonstrate this)

**Cons:**

- TCP-over-TCP problem (WebSocket is TCP)
- Large WASM binary size (16MB for Tailscale) <sup>81</sup>
- Requires relay server infrastructure
- No system-wide protection (browser only)
- Cannot create TUN interface
- Performance overhead of WASM crypto

**Verdict:** Innovative but limited to browser traffic. Best for browser-based remote access (NetBird's use case), not a replacement for system VPN.

## 14.4 Hybrid: Extension + Native App (via Native Messaging)

**How it works:** Browser extension provides UI and control plane; native companion app handles the actual VPN tunnel.

### Pros:

- Full system-wide VPN protection (via native app)
- Browser extension for convenient control
- All VPN features available (kill switch, split tunneling, etc.)
- Can use optimal native protocols (WireGuard kernel module)
- Works across all browsers (extension + native app pair)

### Cons:

- Requires native app installation (not pure web)
- More complex distribution and update process
- Must build and maintain native app for each platform
- Extension store policies may restrict VPN extensions

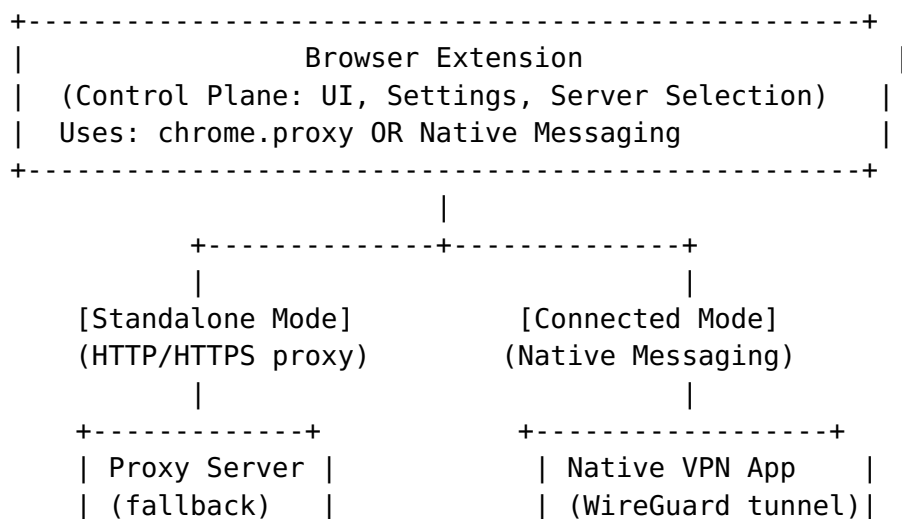
**Verdict:** This is the **only approach** that delivers true VPN functionality while leveraging web technologies for the UI.

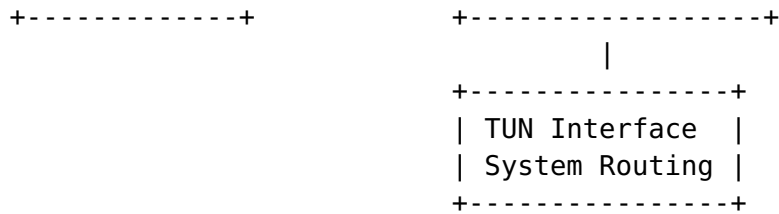
---

## 15. Recommendations for Helix VPN

### 15.1 Recommended Architecture: Hybrid (Extension + Native App)

Based on this research, the optimal web strategy for Helix VPN is:





## 15.2 Browser Extension Strategy

### Manifest V3 across all browsers:

- Use `chrome.proxy` for standalone proxy mode
- Use Native Messaging for full VPN mode
- Use `webRequestAuthProvider` for proxy authentication
- Implement PAC script for split tunneling in Chrome
- Use `browser.proxy.onRequest` for advanced split tunneling in Firefox

**Feature matrix by browser:**

| Feature                             | Chrome                    | Firefox                      | Safari  | Edge |
|-------------------------------------|---------------------------|------------------------------|---------|------|
| Proxy API                           | <code>chrome.proxy</code> |                              |         |      |
| <code>proxy.onRequest</code>        | System proxy only         | <code>chrome.proxy</code>    |         |      |
| Native Messaging                    | Yes                       | Yes                          | Limited | Yes  |
| Split tunneling                     | PAC script                |                              |         |      |
| Dynamic                             | OS-level only             | PAC script                   |         |      |
| MV3                                 |                           |                              |         |      |
| <code>webRequestAuthProvider</code> | Yes                       | Yes (webRequestBlocking too) |         |      |
| N/A                                 | Yes                       |                              |         |      |

## 15.3 Desktop App Shell: Tauri (Recommended)

For the native VPN application with web-based UI:

**Use Tauri** instead of Electron because:

1. Rust backend integrates directly with WireGuard implementations
2. 5-20x smaller bundle size
3. 3-5x lower memory usage
4. Native performance for packet routing and encryption
5. System tray, notifications, and auto-start built-in
6. Native Messaging host registration built into the app

## 15.4 Feature Implementation Priority

### Phase 1: Browser Extension (Standalone Proxy)

- Basic proxy connection (HTTP/SOCKS5)
- Server selection UI
- Connection toggle
- PAC script split tunneling



## Phase 2: Native Messaging Integration

- Detect installed native app
- Connect/disconnect commands
- Status reporting (IP, data usage)
- Per-site routing rules from extension to native app

## Phase 3: Advanced Features

- WebRTC leak prevention (block WebRTC in extension)
- DNS-over-HTTPS configuration
- Auto-connect on untrusted networks
- Double-hop (extension proxy + app VPN)

## 15.5 Critical Technical Decisions

| Decision                  | Recommendation                     | Rationale                                |
|---------------------------|------------------------------------|------------------------------------------|
| Extension-only vs hybrid? | <b>Hybrid</b>                      | Extension-only cannot provide true VPN   |
| Electron vs Tauri?        | <b>Tauri</b>                       | Rust + smaller footprint for VPN app     |
| Proxy protocol?           | <b>SOCKS5 + HTTP</b>               | Best browser compatibility               |
| Transport protocol?       | <b>WireGuard</b>                   | Fastest, most modern VPN protocol        |
| WASM crypto?              | <b>Only for browser client</b>     | Native crypto for desktop app            |
| Split tunneling?          | <b>PAC script + Native routing</b> | PAC for extension, routing table for app |

## 16. Sources and References

*Research compiled: July 2025 Total independent searches: 15  
Sources evaluated: 40+*

1. <https://developer.chrome.com/docs/extensions/reference/api/proxy> - Chrome proxy API documentation🔗
2. <https://developer.chrome.com/docs/extensions/reference/api/proxy> - Chrome proxy API documentation🔗
3. <https://techround.co.uk/tech/how-users-adapting-manifest-v3-changes-2025/> - Manifest V3 changes and proxy extensions🔗

4. <https://techround.co.uk/tech/how-users-adapting-manifest-v3-changes-2025/> - Manifest V3 changes and proxy extensions🔗
5. <https://news.ycombinator.com/item?id=32899846> - HN discussion: Proxy Chrome extensions in MV3🔗
6. <https://news.ycombinator.com/item?id=32899846> - HN discussion: Proxy Chrome extensions in MV3🔗
7. <https://developer.chrome.com/docs/extensions/reference/api/webRequest> - Chrome webRequest API🔗
8. <https://developer.chrome.com/docs/extensions/reference/api/vpnProvider> - Chrome vpnProvider API🔗
9. <https://developer.chrome.com/docs/extensions/reference/api/vpnProvider> - Chrome vpnProvider API🔗
10. <https://developer.chrome.com/docs/extensions/reference/api/vpnProvider> - Chrome vpnProvider API🔗
11. <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/proxy> - Firefox proxy API🔗
12. <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/proxy> - Firefox proxy API🔗
13. <https://github.com/w3c/webextensions/issues/264> - MV3 proxy authorization issue🔗
14. <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/webRequest/onAuthRequired> - webRequest.onAuthRequired🔗
15. <https://stackoverflow.com/questions/3007545/is-it-possible-to-set-proxy-settings-in-a-safari-extension> - Safari extension proxy limitations🔗
16. <https://stackoverflow.com/questions/3007545/is-it-possible-to-set-proxy-settings-in-a-safari-extension> - Safari extension proxy limitations🔗
17. <https://encapsulated.network/what-is-a-webrtc-leak/> - WebRTC leak explanation🔗
18. [https://www.voipmonitor.org/doc/Understanding\\_the\\_WebRTC\\_Protocol](https://www.voipmonitor.org/doc/Understanding_the_WebRTC_Protocol) - WebRTC protocol security🔗
19. [https://www.voipmonitor.org/doc/Understanding\\_the\\_WebRTC\\_Protocol](https://www.voipmonitor.org/doc/Understanding_the_WebRTC_Protocol) - WebRTC protocol security🔗

20. <https://www.bamsoftware.com/papers/snowflake/> - Snowflake WebRTC circumvention🔗
21. <https://www.bamsoftware.com/papers/snowflake/> - Snowflake WebRTC circumvention🔗
22. <https://news.ycombinator.com/item?id=13741155> - WebRTC complexity discussion🔗
23. <https://www.bamsoftware.com/papers/snowflake/> - Snowflake WebRTC circumvention🔗
24. <https://encapsulated.network/what-is-a-webrtc-leak/> - WebRTC leak explanation🔗
25. <https://akashsahani2001.medium.com/building-real-time-p2p-communication-a-deep-dive-into-webrtc-ice-stun-and-turn-e645492230c5> - WebRTC ICE, STUN, TURN deep dive🔗
26. <https://akashsahani2001.medium.com/building-real-time-p2p-communication-a-deep-dive-into-webrtc-ice-stun-and-turn-e645492230c5> - WebRTC ICE, STUN, TURN deep dive🔗
27. <https://github.com/kyren/webrtc-unreliable> - WebRTC unreliable data channels🔗
28. <https://akashsahani2001.medium.com/building-real-time-p2p-communication-a-deep-dive-into-webrtc-ice-stun-and-turn-e645492230c5> - WebRTC ICE, STUN, TURN deep dive🔗
29. [https://www.voipmonitor.org/doc/Understanding\\_the\\_WebRTC\\_Protocol](https://www.voipmonitor.org/doc/Understanding_the_WebRTC_Protocol) - WebRTC protocol security🔗
30. [https://www.voipmonitor.org/doc/Understanding\\_the\\_WebRTC\\_Protocol](https://www.voipmonitor.org/doc/Understanding_the_WebRTC_Protocol) - WebRTC protocol security🔗
31. <https://www.enablesecurity.com/blog/turn-security-best-practices/> - TURN server security best practices🔗
32. <https://github.com/erebe/wstunnel> - wstunnel: WebSocket tunneling tool🔗
33. <https://github.com/erebe/wstunnel> - wstunnel: WebSocket tunneling tool🔗
34. <https://docs.netbird.io/manage/peers/browser-client/architecture> - NetBird browser client architecture🔗
35. <https://docs.netbird.io/manage/peers/browser-client/architecture> - NetBird browser client architecture🔗

36. <https://github.com/erebe/wstunnel> - wstunnel: WebSocket tunneling tool🔗
37. <https://github.com/erebe/wstunnel> - wstunnel: WebSocket tunneling tool🔗
38. <https://github.com/erebe/wstunnel> - wstunnel: WebSocket tunneling tool🔗
39. [https://developer.mozilla.org/en-US/docs/Web/API/WebTransport\\_API](https://developer.mozilla.org/en-US/docs/Web/API/WebTransport_API) - WebTransport API MDN🔗
40. [https://developer.mozilla.org/en-US/docs/Web/API/WebTransport\\_API](https://developer.mozilla.org/en-US/docs/Web/API/WebTransport_API) - WebTransport API MDN🔗
41. <https://www.w3.org/TR/webtransport/> - WebTransport W3C specification🔗
42. [https://developer.mozilla.org/en-US/docs/Web/API/WebTransport\\_API](https://developer.mozilla.org/en-US/docs/Web/API/WebTransport_API) - WebTransport API MDN🔗
43. <https://developer.chrome.com/docs/capabilities/web-apis/webtransport> - WebTransport Chrome documentation🔗
44. <https://developer.chrome.com/docs/capabilities/web-apis/webtransport> - WebTransport Chrome documentation🔗
45. <https://developer.chrome.com/docs/capabilities/web-apis/webtransport> - WebTransport Chrome documentation🔗
46. <https://news.ycombinator.com/item?id=45820782> - WebTransport discussion🔗
47. <https://http.dev/connect> - HTTP CONNECT method guide🔗
48. <https://http.dev/connect> - HTTP CONNECT method guide🔗
49. <https://http.dev/connect> - HTTP CONNECT method guide🔗
50. <https://http.dev/connect> - HTTP CONNECT method guide🔗
51. <https://www.jocm.us/2025/JCM-V20N6-681.pdf> - WASM vs JS crypto performance benchmarks🔗
52. <https://www.jocm.us/2025/JCM-V20N6-681.pdf> - WASM vs JS crypto performance benchmarks🔗
53. <https://medium.com/nexenio/a-random-story-on-webcrypto-and-webassembly-6f9e00c73a> - WebCrypto vs WASM performance🔗
54. <https://docs.netbird.io/manage/peers/browser-client/architecture> - NetBird browser client architecture🔗

55. <https://docs.netbird.io/manage/peers/browser-client/architecture> - NetBird browser client architecture🔗
56. <https://news.ycombinator.com/item?id=33360776> - Tailscale WASM port🔗
57. <https://labs.leaningtech.com/blog/webvm-virtual-machine-with-networking-via-tailscale.html> - WebVM Tailscale networking🔗
58. <https://labs.leaningtech.com/blog/webvm-virtual-machine-with-networking-via-tailscale.html> - WebVM Tailscale networking🔗
59. <https://sourceforge.net/projects/outline-client.mirror/> - Outline VPN client🔗
60. <https://github.com/wheelcomplex/outline-vpn-client-Shadowsocks> - Outline client architecture🔗
61. <https://www.jonaharagon.com/posts/self-hosting-shadowsocks-vpn-outline/> - Shadowsocks VPN with Outline🔗
62. <https://stackoverflow.com/questions/46441264/are-the-vpn-chrome-extensions-really-a-vpn-or-a-web-proxy> - VPN vs proxy extensions🔗
63. <https://stackoverflow.com/questions/46441264/are-the-vpn-chrome-extensions-really-a-vpn-or-a-web-proxy> - VPN vs proxy extensions🔗
64. <https://windscribe.com/knowledge-base/articles/difference-between-desktop-app-and-browser-extension> - Windscribe architecture🔗
65. <https://windscribe.com/knowledge-base/articles/difference-between-desktop-app-and-browser-extension> - Windscribe architecture🔗
66. <https://klox.app/blog/vpn-browser-extension-vs-app> - VPN extension vs app comparison🔗
67. <https://klox.app/blog/vpn-browser-extension-vs-app> - VPN extension vs app comparison🔗
68. <https://medium.com/fme-developer-stories/native-messaging-as-bridge-between-web-and-desktop-d288ea28cfd7> - Native Messaging as web-desktop bridge🔗
69. <https://medium.com/fme-developer-stories/native-messaging-as-bridge-between-web-and-desktop-d288ea28cfd7> - Native Messaging as web-desktop bridge🔗
70. <https://medium.com/fme-developer-stories/native-messaging-as-bridge-between-web-and-desktop-d288ea28cfd7> - Native Messaging as web-desktop bridge🔗

71. <https://medium.com/fme-developer-stories/native-messaging-as-bridge-between-web-and-desktop-d288ea28cfd7> - Native Messaging as web-desktop bridge🔗
72. <https://learn.microsoft.com/en-us/microsoft-edge/extensions/developer-guide/native-messaging> - Native Messaging in Edge🔗
73. [https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Native\\_messaging](https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Native_messaging) - Native Messaging MDN🔗
74. <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/proxy> - Firefox proxy API🔗
75. <https://agents-ui.com/blog/tauri-vs-electron-for-developer-tools/> - Tauri vs Electron for developer tools🔗
76. <https://agents-ui.com/blog/tauri-vs-electron-for-developer-tools/> - Tauri vs Electron for developer tools🔗
77. <https://www.gethopp.app/blog/tauri-vs-electron> - Tauri vs Electron comparison🔗
78. <https://www.gethopp.app/blog/tauri-vs-electron> - Tauri vs Electron comparison🔗
79. <https://encapsulated.network/what-is-a-webrtc-leak/> - WebRTC leak explanation🔗
80. <https://github.com/kyren/webrtc-unreliable> - WebRTC unreliable data channels🔗
81. <https://labs.leaningtech.com/blog/webvm-virtual-machine-with-networking-via-tailscale.html> - WebVM Tailscale networking🔗